

NurEngine

Rionaldo Casey Pandhitha
NIM: 13525030

Raffi Fauzi Hermawan
NIM: 13525054

Fachry Azriel Fajdwani
NIM: 13525110



28 September 2026

Bab 1

Pendahuluan

1.1 Latar Belakang

Di pelosok Snezhnaya, seorang peneliti muda bernama Mitya tengah berusaha keras memimpin eksperimen krusial berskala raksasa: Proyek Stuzha. Proyek ini digadang-gadang mampu mengendalikan fluktuasi energi ekstrem di daratan Teyvat. Sayangnya, impian besar tersebut terbentur tembok tebal: Mitya masih mengandalkan kalkulator saku tua yang sering error tiap kali dipaksa menyelesaikan kalkulasi berdimensi tinggi.



Gambar 1.1: Berkenalan dengan Mitya

Setelah menjelajahi perpustakaan Sumeru Akademiya hingga menyisir kabar burung ke seantero Teyvat, Mitya mendengar reputasi kalian sebagai praktisi komputasi dan aljabar linier paling piawai. Ia pun mengirimkan pesan ke Adventures Guild dan memberikan komisi perangkat lunak sekelas Photomath versi Teyvat, yaitu sebuah mesin kalkulasi otomatis yang sanggup melumat sistem matriks dan persamaan linier rumit demi kelangsungan Proyek Stuzha.

Aljabar linier bukan sekadar teori di atas kertas papirus; ini adalah fondasi komputasi modern yang menggerakkan simulasi rekayasa, pemodelan aliran energi, analisis data, hingga dinamika sistem kompleks. Agar kalkulasi Proyek Stuzha tidak runtuh di tengah jalan, kali-an ditugaskan merancang dan membangun pustaka (*library*) komputasi aljabar linier mandiri berbasis bahasa pemrograman Java yang modular, tangguh, dan terdokumentasi rapi.

Pustaka ini nantinya menjadi motor utama di balik alat komputasi Mitya untuk:

- Menyelesaikan Sistem Persamaan Linier (SPL), perhitungan Determinan, dan pencarian Invers Matriks guna menganalisis keseimbangan sirkuit daya dan kestabilan instrumen Proyek Stuzha.
- Melakukan Interpolasi Polinomial, Natural Cubic Spline Interpolation, dan regresi Spline untuk memetakan jalur transfer energi serta memodelkan kontur data eksperimen secara presisi.
- Menerapkan *Image Inpainting* (Hole Filling) berbasis Persamaan Laplace (Bonus) sebagai modul restorasi darurat. Modul ini bertugas merekonstruksi lembar cetak biru dan catatan eksperimen penting yang kerap berlubang atau hilang secara misterius yang konon merupakan ulah sabotase usil dari Renova.

Melalui tugas besar ini, kalian diharapkan tidak hanya menguasai teori matematika di balik algoritma-algoritma tersebut, melainkan juga mampu mengonstruksinya menjadi perangkat lunak nyata yang lolos berbagai skenario uji (*test cases*) ekstrem.

1.2 Tujuan Tugas Besar

- Mengimplementasikan metode penyelesaian SPL, determinan, invers matriks, interpolasi polinomial, Natural Cubic Spline Interpolation, regresi Spline, dan algoritma *Image Inpainting* berbasis diskritisasi Persamaan Laplace secara mandiri menggunakan Java.
- Membangun pustaka (*library*) aljabar linier yang terstruktur secara modular, bersih, dan terdokumentasi dengan baik. Tidak menggunakan *library* perhitungan matriks, regresi, atau interpolasi instan.
- Menguji keandalan pustaka pada berbagai kasus uji analitis dan restorasi citra, serta menyusun laporan analisis hasil perhitungannya.

Bab 2

Dasar Teori

Bab ini memuat teori yang digunakan pada seluruh metode yang diimplementasikan dalam program.

2.1 Sistem Persamaan Linier

Sistem Persamaan Linear (SPL) adalah himpunan berhingga dari persamaan-persamaan linear dengan variabel-variabel yang sama. Bentuk umum dari SPL dengan m persamaan dan n variabel adalah

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n &= b_1, \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n &= b_2, \\ &\vdots \\ a_{m1}x_1 + a_{m2}x_2 + a_{m3}x_3 + \cdots + a_{mn}x_n &= b_m. \end{aligned}$$

SPL dapat dinyatakan dalam bentuk $A\mathbf{x} = \mathbf{b}$ maupun dalam bentuk matriks *augmented*. [9, 2]

Tiga operasi baris elementer terhadap matriks *augmented* adalah:

1. Kalikan sebuah baris dengan konstanta tidak nol.
2. Pertukarkan dua buah baris.
3. Tambahkan sebuah baris dengan kelipatan baris lainnya.

Solusi sebuah SPL diperoleh dengan menerapkan OBE sampai terbentuk *row echelon form* (REF) atau *reduced row echelon form* (RREF). Jika berakhir pada REF digunakan metode eliminasi Gauss, sedangkan jika berakhir pada RREF digunakan metode eliminasi Gauss-Jordan. [2]

REF memiliki 1 utama (*leading one*) pada setiap baris yang tidak seluruhnya nol, seluruh baris nol berada di bagian bawah, dan 1 utama pada baris yang lebih rendah terletak lebih jauh ke kanan. Kolom di bawah 1 utama bernilai nol. RREF memiliki sifat REF dan setiap kolom yang memiliki 1 utama juga bernilai nol di tempat lain. [1]

Ada tiga kemungkinan solusi yang dapat terjadi pada SPL:

1. mempunyai solusi yang unik atau tunggal,
2. mempunyai banyak solusi atau tidak berhingga banyaknya, atau
3. tidak memiliki solusi sama sekali.

Solusi banyak dapat dituliskan dalam bentuk parametrik, sedangkan sistem tidak memiliki solusi jika eliminasi menghasilkan persamaan yang tidak mungkin dipenuhi. [3]

2.1.1 Eliminasi Gauss

Metode eliminasi Gauss menyelesaikan SPL dengan melakukan Operasi Baris Elementer (OBE) pada matriks *augmented* sampai diperoleh *row echelon form* (REF). Pada REF, elemen-elemen di bawah setiap pivot bernilai nol sehingga sistem persamaan dapat diselesaikan secara bertahap dari persamaan terakhir menuju persamaan pertama. [2]

Langkah-langkah metode eliminasi Gauss adalah:

1. Nyatakan SPL dalam bentuk matriks *augmented*.
2. Terapkan OBE pada matriks *augmented* sampai terbentuk REF.
3. Selesaikan persamaan yang berkoresponden pada REF dengan teknik penyulihan mundur (*backward substitution*).

[2]

Untuk menjaga stabilitas numerik, program menerapkan *partial pivoting*, yaitu memilih elemen dengan nilai absolut terbesar pada kolom yang sedang diproses sebagai pivot sebelum proses eliminasi. [9]

2.1.2 Eliminasi Gauss-Jordan

Metode eliminasi Gauss-Jordan merupakan pengembangan dari metode eliminasi Gauss. OBE diterapkan pada matriks *augmented* sampai diperoleh *reduced row echelon form* (RREF). Pada RREF, setiap pivot bernilai 1 dan seluruh elemen lain pada kolom pivot bernilai nol. Oleh karena itu, jika SPL memiliki solusi unik, nilai variabel dapat langsung diperoleh dari matriks akhir tanpa melakukan penyulihan mundur. [4]

Metode eliminasi Gauss-Jordan terdiri atas dua fase:

1. Fase maju (*forward phase*), yaitu proses eliminasi Gauss untuk menghasilkan nilai nol di bawah pivot.
2. Fase mundur (*backward phase*), yaitu menghasilkan nilai nol di atas pivot hingga terbentuk RREF.

Kedua fase tersebut dapat dilakukan secara bersamaan maupun secara sekuensial. [4]

Pada implementasi program, proses pembentukan RREF dilanjutkan dari REF dengan menormalkan pivot menjadi 1 dan mengeliminasi seluruh elemen di atas pivot.

2.1.3 Metode Matriks Balikan

Untuk SPL

$$A\mathbf{x} = \mathbf{b},$$

jika matriks A memiliki invers, solusi dapat diperoleh dengan

$$\mathbf{x} = A^{-1}\mathbf{b}.$$

Sebuah matriks hanya memiliki invers jika $\det(A) \neq 0$. [9]

Pada program, matriks A dan vektor $\mathbf{b}$ dipisahkan dari matriks *augmented*, invers A dihitung, kemudian hasilnya dikalikan dengan $\mathbf{b}$.

2.1.4 Kaidah Cramer

Kaidah Cramer digunakan untuk SPL dengan jumlah persamaan sama dengan jumlah variabel dan $\det(A) \neq 0$. Jika A_j adalah matriks yang diperoleh dengan mengganti kolom ke- j dari matriks A dengan vektor $\mathbf{b}$, maka

$$x_j = \frac{\det(A_j)}{\det(A)}, \quad j = 1, 2, \dots, n.$$

[9]

Pada program, determinan A dihitung terlebih dahulu, kemudian setiap A_j dibentuk dan determinannya digunakan untuk memperoleh nilai masing-masing variabel.

2.2 Determinan Matriks

Determinan adalah nilai skalar yang dapat dihitung dari elemen-elemen sebuah matriks persegi. Determinan dari matriks A dinotasikan sebagai $\det(A)$ atau $|A|$. [9]

Secara umum, untuk matriks segitiga A berukuran $n \times n$,

$$\det(A) = a_{11}a_{22}a_{33} \cdots a_{nn}.$$

[5]

2.2.1 Metode Reduksi Baris

Jika sebuah baris dikalikan dengan k , determinan ikut dikalikan dengan k . Jika dua baris dipertukarkan, tanda determinan berubah. Jika sebuah baris ditambahkan dengan k kali baris lain, nilai determinan tidak berubah. [5]

Pada program, matriks direduksi ke REF sehingga berbentuk segitiga. Determinan diperoleh dari hasil kali elemen diagonal dan tanda hasil disesuaikan dengan banyaknya pertukaran baris.

2.2.2 Metode Ekspansi Kofaktor

Untuk matriks A berukuran $n \times n$, M_{ij} adalah minor entri a_{ij} , yaitu determinan upa-matriks yang tidak memuat baris i dan kolom j . Kofaktor entri a_{ij} didefinisikan sebagai

$$C_{ij} = (-1)^{i+j} M_{ij}.$$

[6]

Determinan dapat dihitung secara baris atau kolom:

$$\det(A) = \sum_{j=1}^n a_{ij} C_{ij}$$

atau

$$\det(A) = \sum_{i=1}^n a_{ij} C_{ij}.$$

Baris atau kolom dengan banyak elemen nol dapat dipilih untuk mengurangi jumlah perhitungan. [6]

2.3 Matriks Balikan

Invers dari sebuah matriks persegi A adalah matriks A^{-1} yang memenuhi

$$AA^{-1} = A^{-1}A = I.$$

Matriks hanya memiliki invers jika determinannya tidak nol. Perhitungan invers dapat dilakukan dengan metode augmentasi Gauss-Jordan maupun metode adjoin. [9]

2.3.1 Metode Augmentasi

Matriks A digabungkan dengan matriks identitas menjadi

$$[A \mid I].$$

Gauss-Jordan kemudian digunakan sampai bagian kiri menjadi identitas:

$$[A \mid I] \sim [I \mid A^{-1}].$$

Pada implementasi program, matriks *augmented* tersebut diubah ke RREF dan bagian kanan diambil sebagai A^{-1} .

2.3.2 Metode Adjoin

Adjoin dari A adalah transpose matriks kofaktor:

$$\text{adj}(A) = C^T.$$

Balikan matriks A dihitung dengan

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A).$$

[6]

Program mengikuti alur tersebut dengan menghitung determinan dan adjoin, lalu mengalikan setiap elemen adjoin dengan $1/\det(A)$.

2.4 Interpolasi Polinomial

Bila fungsi cocok yang digunakan berbentuk polinom, polinom tersebut dinamakan polinom interpolasi. Pekerjaan menginterpolasi titik data dengan sebuah polinom disebut interpolasi dengan polinom. [8]

Diberikan $n + 1$ titik berbeda

$$(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n),$$

dicari polinom $p_n(x)$ yang melewati seluruh titik tersebut sehingga

$$y_i = p_n(x_i), \quad i = 0, 1, 2, \dots, n.$$

[8]

Secara umum,

$$P_n(x) = a_0 + a_1x + a_2x^2 + \cdots + a_nx^n.$$

Koefisien $a_0, a_1, \dots, a_n$ dapat dicari dengan menyulihkan seluruh titik data ke dalam polinom sehingga terbentuk SPL. [9]

Pada implementasi program, SPL tersebut disusun dalam bentuk matriks dengan kolom $1, x, x^2, \dots, x^n$, kemudian koefisien polinom diperoleh dengan menyelesaikannya menggunakan RREF.

2.5 Natural Cubic Spline Interpolation

Interpolasi Spline Kubik adalah metode interpolasi yang mengonstruksi kurva mulus melalui sekumpulan titik data menggunakan potongan polinomial kubik pada setiap interval. Kurva dibuat kontinu hingga turunan kedua. [9]

Untuk *natural cubic spline*, nilai fungsi, turunan pertama, dan turunan kedua harus kontinu pada simpul interior, serta berlaku

$$S''(x_0) = S''(x_n) = 0.$$

Syarat batas tersebut menghasilkan sistem linear tridiagonal untuk menentukan nilai turunan kedua pada titik-titik data. [9, 11]

Alur implementasi mengikuti proses tersebut. Titik data terlebih dahulu diurutkan berdasarkan absis. Program membentuk SPL tridiagonal hanya untuk titik interior, sedangkan nilai turunan kedua pada kedua ujung dibiarkan nol. SPL tersebut diselesaikan dengan REF. Nilai interpolasi kemudian dihitung menggunakan segmen yang memuat nilai x yang dicari. Alur pengurutan titik, penyelesaian nilai knot interior, dan evaluasi per segmen juga sejalan dengan referensi yang digunakan pada kode. [10]

2.6 Regresi Spline Kubik

Splines Regression memodelkan hubungan non-linear antara variabel prediktor x dan respons y menggunakan potongan polinomial yang terhubung pada *knots*. Pada *truncated power basis*, fungsi regresi dapat ditulis sebagai

$$f(x) = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3 + \sum_{k=1}^K \beta_{3+k}(x - \xi_k)_+^3,$$

dengan

$$(u)_+ = \max(0, u).$$

[9]

Untuk N data dan K knots, program membentuk matriks desain X dengan kolom

$$1, x, x^2, x^3, (x - \xi_1)_+^3, \dots, (x - \xi_K)_+^3.$$

Koefisien dicari menggunakan Persamaan Normal

$$(X^T X)\hat{\beta} = X^T y.$$

Persamaan normal merupakan bentuk standar untuk menyelesaikan masalah kuadrat terkecil melalui sistem linear. [9, 12]

Pada implementasi program, $X^T X$ dan $X^T y$ dihitung terlebih dahulu, digabungkan menjadi SPL, direduksi ke REF untuk membantu normalisasi nilai numerik, lalu koefisien akhirnya diperoleh melalui RREF. Jumlah data juga harus memenuhi $N \geq K + 4$.

2.7 Image Inpainting dengan Persamaan Laplace

Image inpainting digunakan untuk merekonstruksi area citra yang ditandai oleh *mask*. Pada spesifikasi tugas, piksel putih pada *mask* merupakan piksel yang perlu diestimasi, sedangkan piksel lainnya menjadi bagian citra yang dipertahankan. Nilai piksel yang hilang dimodelkan dengan diskritisasi Persamaan Laplace:

$$4f_{i,j} - f_{i-1,j} - f_{i+1,j} - f_{i,j-1} - f_{i,j+1} = 0.$$

Dengan demikian, nilai sebuah piksel pada daerah yang diisi dapat diperoleh dari rata-rata tetangga atas, bawah, kiri, dan kanan. [9, 13]

Implementasi program membaca gambar asli dan *mask*, memeriksa format, kesamaan ukuran, dan batas maksimum 512×512 piksel, lalu memetakan piksel putih (255, 255, 255) sebagai *hole*. Kanal merah, hijau, dan biru diproses secara terpisah. Nilai awal pada piksel *hole* dibuat nol, kemudian piksel diperbarui berurutan menggunakan rata-rata tetangga yang masih berada di dalam citra. Nilai baru langsung digunakan untuk perhitungan piksel berikutnya sehingga proses tersebut mengikuti prinsip Gauss-Seidel. [7]

Iterasi berhenti ketika perubahan absolut maksimum dari seluruh kanal dan seluruh piksel *hole* sudah tidak melebihi toleransi 10^{-5} . Setelah konvergen, nilai RGB hasil iterasi dibulatkan, digabungkan kembali dengan piksel asli yang tidak di-*mask*, lalu gambar disimpan sebagai keluaran. Secara matematis proses ini ekuivalen dengan menyelesaikan SPL dari diskritisasi Laplace, tetapi implementasi tidak membentuk matriks $N \times N$ secara eksplisit.

2.7.1 Metode Gauss-Seidel

Pada metode Gauss-Seidel, nilai baru yang sudah dihitung langsung dipakai untuk menghitung komponen berikutnya. [7] Secara umum,

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right).$$

Urutan pembaruan berpengaruh pada Gauss-Seidel karena komponen yang telah diperbarui segera digunakan pada iterasi yang sama. [14]

Bab 3

Implementasi

3.1 Struktur

Terdapat 7 berkas (2 berkas utama dan 5 berkas modul) dengan rincian sebagai berikut.

- `Matrix.java`: Memuat fungsi adisi dan multiplikasi matriks, pengambilan submatriks, pembentukan matriks kofaktor, operasi transpose matriks, pembentukan matriks adjoin, fungsi Operasi Baris Elementer, fungsi pembentukan matriks augmented, fungsi pembentukan matriks eselon baris dan reduksinya, beserta perhitungan determinan dan pembentukan matriks invers. Terdapat juga konstanta yang digunakan untuk perhitungan/komparasi dan normalisasi nilai.
- `SPLSolver.java`: Menyediakan 4 metode penyelesaian SPL sesuai spesifikasi, yakni Eliminasi Gauss, Eliminasi Gauss-Jordan, Kaidah Cramer, dan Invers.
- `Interpolation.java`: Menyediakan 2 jenis interpolasi sesuai spesifikasi, yakni Polinomial dan Natural Cubic Spline beserta fungsi evaluasinya.
- `Regression.java`: Menyediakan regresi Cubic Spline dengan Truncated Power Method beserta fungsi evaluasinya.
- `ImageHoleFill.java`: Memuat fungsi yang dapat mengonstruksi gambar rusak dengan mask dari luar menggunakan algoritma Gauss-Seidel. Algoritma ditanam secara langsung ke fungsi pembenahan, tidak diabstraksikan seperti yang ada di `Matrix.java`.
- `App.java`: Memuat segala hal yang berhubungan dengan I/O, terutama printing dan formatting matriks, persamaan, dan solusi atas SPL, beserta tampilan dan alur CLI.
- `GUI.java`: Versi GUI dari `App.java` menggunakan JavaFX. Fungsi-fungsi tetap persis sama.

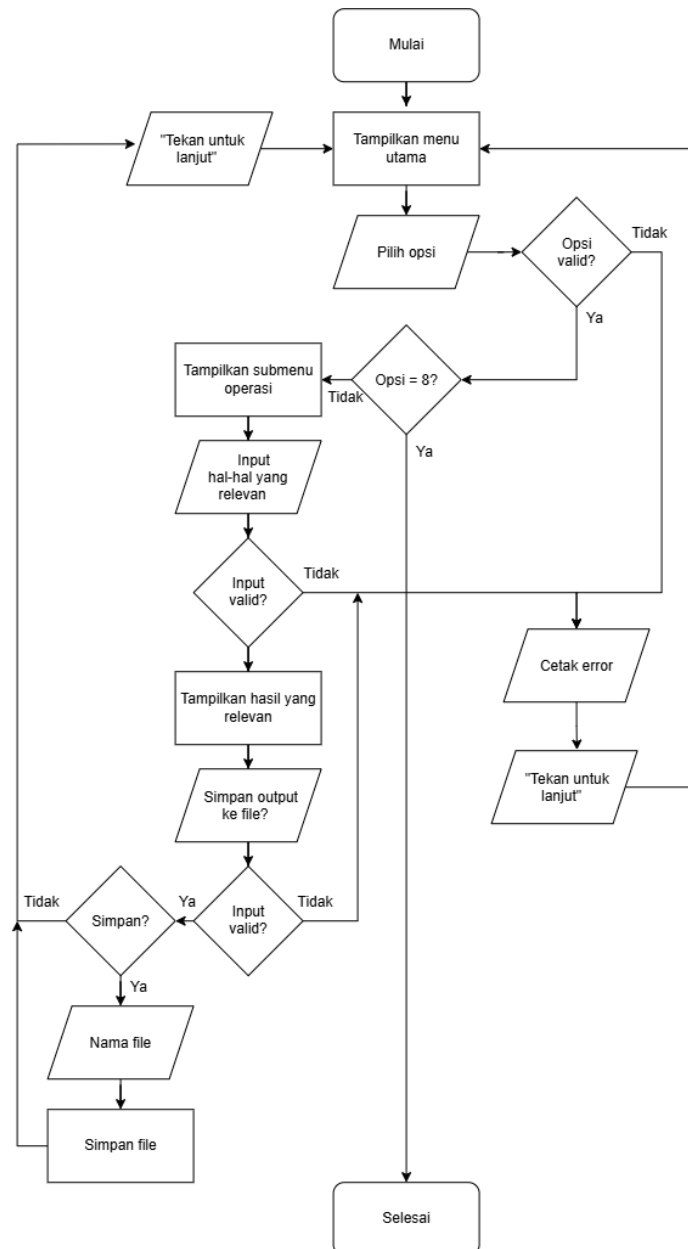
Selain itu, program ini tidak memiliki tipe data bentukan. Input matriks, point, dan output hasil SPL Solver semuanya menggunakan `double[][]`.

3.2 Building

Maven digunakan untuk membangun bytecode keseluruhan program secara otomatis. Perintah `mvn clean package` akan menghasilkan folder baru bernama 'target'. Folder tersebut

terdiri atas bytecode dalam bentuk `.jar` dan segala berkas-berkas yang relevan, sehingga ada 2 opsi untuk menjalankan program CLI: `mvn exec:java` ataupun `java -jar [file.jar]`. Juga tersedia opsi untuk menjalankan program GUI: `mvn javafx:run`.

3.3 Alur program



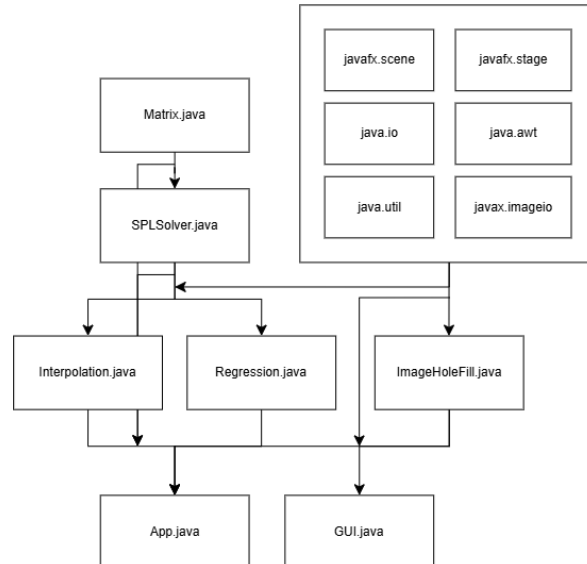
Gambar 3.1: Garis Besar Alur Penggunaan

3.4 Arsitektur

Modul-modul yang digunakan dibuat sedemikian rupa sehingga memiliki dependensi yang jelas. Modul `Matrix.java` dan `ImageHoleFill.java` tidak memiliki dependensi sama sekali. Modul `SPLSolver.java` bergantung pada `Matrix.java`. Modul yang tersisa, yakni

Interpolation.java dan Regression.java bergantung pada SPLSolver, secara tidak langsung juga bergantung pada Matrix.java.

Alasan ImageHoleFill.java tidak bergantung pada Matrix.java adalah karena perlakuan dan algoritmanya yang sangat berbeda. Selain itu, hal ini memungkinkan kedua modul dikerjakan dalam waktu yang bersamaan untuk meningkatkan efisiensi.



Gambar 3.2: Jalur Dependensi

Bab 4

Eksperimen

Bab ini memaparkan hasil eksekusi program terhadap seluruh kasus uji pada Bab 5 dokumen [Spesifikasi Tugas Besar](#). Setiap kasus diuji dengan metode yang relevan sesuai spesifikasi. Pengujian dilakukan melalui masukan manual dan/atau berkas .txt. Bagian ini memuat input, output, tangkapan layar bila diperlukan, serta analisis singkat untuk setiap kasus.

4.1 Determinan Matriks

4.1.1 Kasus Uji 1

Input: Matriks dari berkas test\det_kasus-1.txt.

$$A = \begin{bmatrix} 0 & 2 & -1 & 3 \\ 4 & 1 & 2 & -2 \\ -2 & 3 & 0 & 1 \\ 1 & -1 & 5 & 2 \end{bmatrix}$$

Tabel 4.1: Hasil pengujian determinan Kasus Uji 1

Metode	Nilai Determinan	Status
Ekspansi Kofaktor	-259.000	Valid
Reduksi Baris (OBE)	-259.000	Valid

Analisis: Kedua metode menghasilkan -259 , sama persis dengan perhitungan eksak. Karena $\det A \neq 0$, matriks tidak singular sehingga dapat dipakai pada invers dan SPL. Pada ordo 4 perbedaan waktu kedua metode tidak terasa, tetapi ekspansi kofaktor berkompleksitas $O(n!)$ sedangkan reduksi baris $O(n^3)$, sehingga reduksi baris jauh lebih layak untuk ordo besar.

4.1.2 Kasus Uji 2

Input: Matriks dari berkas test\det_kasus-2.txt.

$$A = \begin{bmatrix} 2 & -1 & 3 & 0 & 4 \\ 1 & 2 & -2 & 5 & -1 \\ 3 & 1 & 1 & 2 & 0 \\ 0 & 4 & -1 & 3 & 2 \\ 5 & 0 & 4 & 2 & 4 \end{bmatrix}$$

Tabel 4.2: Hasil pengujian determinan Kasus Uji 2

Metode	Nilai Determinan	Status
Ekspansi Kofaktor	0.000	valid
Reduksi Baris (OBE)	-0.000	valid

Analisis: Determinan bernilai 0, sehingga matriks singular ($\text{rank } 4 < 5$, ada baris yang bergantung linear). Nilai “-0.000” pada reduksi baris merupakan galat pembulatan double (orde 10^{-14}) yang tampil sebagai nol bertanda negatif, sedangkan ekspansi kofaktor menghasilkan 0 tepat karena hanya menjumlahkan perkalian bilangan bulat. Perbedaan tanda ini tidak mengubah kesimpulan, dan menunjukkan perlunya ambang toleransi (epsilon) saat menguji “sama dengan nol”.

4.1.3 Kasus Uji 3

Input: Matriks dari berkas `test\det_kasus-3.txt`.

$$A = \begin{bmatrix} \frac{1}{2} & 1 & 0 & \frac{1}{2} \\ \frac{1}{2} & \frac{5}{4} & \frac{1}{4} & \frac{1}{2} \\ -\frac{1}{10} & -\frac{2}{5} & \frac{1}{5} & -\frac{3}{10} \\ \frac{3}{5} & \frac{5}{4} & \frac{9}{20} & \frac{3}{5} \end{bmatrix}$$

Tabel 4.3: Hasil pengujian determinan Kasus Uji 2

Metode	Nilai Determinan	Status
Ekspansi Kofaktor	0.010	valid
Reduksi Baris (OBE)	0.010	valid

Analisis: Entri berupa pecahan sehingga reduksi baris melibatkan pembagian yang tidak eksak dalam biner. Kedua metode tetap menghasilkan nilai yang sama hingga 3 desimal. Determinan bernilai kecil tetapi tidak nol, sehingga matriks tidak singular namun cukup mendekati singular. Karena itu, perbandingan dengan nol harus memakai toleransi yang cukup kecil agar matriks seperti ini tidak salah dianggap singular.

4.2 Invers Matriks

4.2.1 Kasus Uji 1

Input: Matriks dari berkas `test\inverse_kasus1.txt`.

$$A = \begin{bmatrix} 0 & 1 & 2 & -1 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \\ -1 & 2 & 1 & 0 \end{bmatrix}$$

Tabel 4.4: Hasil pengujian invers matriks Kasus Uji 1

Metode	Status
Augment	Valid
Adjoin	Valid

Hasil Augment:

$$A^{-1} = \begin{bmatrix} 0.200 & -0.100 & 0.400 & -0.300 \\ -0.100 & -0.200 & 0.300 & 0.400 \\ 0.400 & 0.300 & -0.200 & -0.100 \\ -0.300 & 0.400 & -0.100 & 0.200 \end{bmatrix}$$

Hasil Adjoin:

$$A^{-1} = \begin{bmatrix} 0.200 & -0.100 & 0.400 & -0.300 \\ -0.100 & -0.200 & 0.300 & 0.400 \\ 0.400 & 0.300 & -0.200 & -0.100 \\ -0.300 & 0.400 & -0.100 & 0.200 \end{bmatrix}$$

Analisis: Kedua metode menghasilkan matriks balikan yang identik hingga 3 desimal dan sesuai dengan $AA^{-1} = I$ ($\det A = 20$). Perbedaan hanya mungkin muncul pada digit ke-15 akibat urutan operasi yang berbeda (galat pembulatan). Metode augment berkompleksitas $O(n^3)$ dan lebih efisien daripada adjoin, yang membutuhkan n^2 kofaktor.

4.2.2 Kasus Uji 2

Input: Matriks dari berkas `test\inverse_kasus2.txt`.

$$A = \begin{bmatrix} 1 & 2 & 0 & -1 & 3 \\ 2 & -1 & 4 & 0 & 1 \\ 0 & 3 & 1 & 2 & -2 \\ 4 & 0 & 5 & 1 & 2 \\ 3 & 1 & 4 & -1 & 4 \end{bmatrix}$$

Tabel 4.5: Hasil pengujian invers matriks Kasus Uji 2

Metode	Hasil	Status
Augment	Matriks balikan tidak ada	Tidak valid (singular)
Adjoin	Matriks balikan tidak ada	Tidak valid (singular)

Analisis: Matriks bersifat *singular* karena $\det A = 0$ ($\text{rank } 4 < 5$), sehingga tidak memiliki invers. Kedua metode mendeteksi kondisi ini dan menampilkan pesan kesalahan tanpa *crash*. Pada metode augment, deteksi bergantung pada ambang epsilon karena pivot yang secara teori bernilai 0 dapat tersisa sebagai sekitar 10^{-15} akibat galat pembulatan.

4.3 Sistem Persamaan Linear $Ax = b$

4.3.1 Kasus Uji 1

Input: Matriks dari `test\spl_kasus-1.txt`.

$$A = \begin{bmatrix} 0 & 2 & -1 & -1 \\ 1 & -1 & 2 & 3 \\ 2 & 1 & 1 & -1 \\ -1 & 3 & 0 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} -5 \\ 15 \\ 1 \\ -3 \end{bmatrix}$$

Tabel 4.6: Hasil SPL $Ax = b$ Kasus Uji 1

Metode	Solusi	Jenis Solusi
Eliminasi Gauss	• $x_0 = 1.000$ • $x_1 = -2.000$ • $x_2 = 3.000$ • $x_3 = 2.000$	tunggal
Eliminasi Gauss-Jordan	• $x_0 = 1.000$ • $x_1 = -2.000$ • $x_2 = 3.000$ • $x_3 = 2.000$	tunggal
Kaidah Cramer	• $x_0 = 1.000$ • $x_1 = -2.000$ • $x_2 = 3.000$ • $x_3 = 2.000$	tunggal
Matriks Balikan	• $x_0 = 1.000$ • $x_1 = -2.000$ • $x_2 = 3.000$ • $x_3 = 2.000$	tunggal

Analisis: Keempat metode menghasilkan solusi tunggal $(x_0, x_1, x_2, x_3) = (1, -2, 3, 2)$ yang memenuhi semua persamaan setelah disubstitusikan kembali. Karena $\det A \neq 0$, Cramer dan Matriks Balikan dapat diterapkan. Gauss dan Gauss-Jordan lebih efisien ($O(n^3)$), sedangkan Cramer membutuhkan $n + 1$ determinan dan matriks balikan membutuhkan pembentukan A^{-1} penuh.

4.3.2 Kasus Uji 2

Input: Matriks dari test\spl_kasus-2.txt.

$$A = \begin{bmatrix} 1 & 2 & -1 & 0 & 3 \\ 2 & 4 & 1 & -1 & 5 \\ -1 & -2 & 2 & 3 & -1 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ 10 \\ -1 \end{bmatrix}$$

Tabel 4.7: Hasil SPL $Ax = b$ Kasus Uji 2

Metode	Solusi	Jenis Solusi
Eliminasi Gauss	• $x_0 = 3.900 - 2.900a_0 - 2.000a_1$ • $x_1 = a_1$ • $x_2 = 1.900 + 0.100a_0$ • $x_3 = -0.300 - 0.700a_0$ • $x_4 = a_0$	tunggal
Eliminasi Gauss-Jordan	• $x_0 = 3.900 - 2.900a_0 - 2.000a_1$ • $x_1 = a_1$ • $x_2 = 1.900 + 0.100a_0$ • $x_3 = -0.300 - 0.700a_0$ • $x_4 = a_0$	tunggal
Kaidah Cramer	Error: SPLSolver.byCramer: Jumlah persamaan tidak sama dengan jumlah variabel!	tunggal
Matriks Balikan	Error: SPLSolver.byInverse: Jumlah persamaan tidak sama dengan jumlah variabel!	tunggal

Analisis: Sistem memiliki 3 persamaan dan 5 variabel dengan $\text{rank}(A) = \text{rank}([A|b]) = 3 < 5$, sehingga solusinya tak hingga dengan $5 - 3 = 2$ parameter bebas (a_0, a_1). Gauss dan Gauss-Jordan menghasilkan bentuk parametrik yang sama. Cramer dan Matriks Balikan menolak sistem ini karena matriks koefisien tidak persegi. Hal ini merupakan keterbatasan metode, bukan berarti sistem tidak memiliki solusi.

4.3.3 Kasus Uji 3

Input: Matriks dari test\spl_kasus-3.txt.

$$A = \begin{bmatrix} 1 & -2 & 3 & 1 \\ 2 & 1 & -1 & 4 \\ 0 & 3 & 2 & -2 \\ 3 & -1 & 2 & 5 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ -1 \\ 3 \\ 4 \end{bmatrix}$$

Tabel 4.8: Hasil SPL $Ax = b$ Kasus Uji 3

Metode	Solusi	Jenis Solusi
Eliminasi Gauss	Error: SPLSolver.byREF: Sistem tidak punya solusi!	tidak ada
Eliminasi Gauss-Jordan	Error: SPLSolver.byRREF: Sistem tidak punya solusi!	tidak ada
Kaidah Cramer	Error: SPLSolver.byCramer: Matriks persamaan tidak mempunyai determinan!	tidak ada
Matriks Balikan	Error: Matrix.inv: determinan matriks bernilai 0	tidak ada

Analisis: Di sini $\text{rank}(A) = 3$ sedangkan $\text{rank}([A|b]) = 4$, sehingga sistem inkonsisten dan tidak memiliki solusi. Gauss dan Gauss-Jordan mendeteksinya melalui baris $[0 \ 0 \ 0 \ 0 | c]$ dengan $c \neq 0$. Cramer dan Matriks Balikan hanya melaporkan $\det A = 0$. Keduanya tidak dapat membedakan “tidak ada solusi” dari “banyak solusi”, sehingga hanya metode eliminasi yang dapat menentukan jenis solusi pada matriks singular.

4.3.4 Kasus Uji 4: Matriks Hilbert

Input: Matriks Hilbert H untuk $n = 6$ dan $n = 10$, dengan $b = [1, 0, \dots, 0]^T$.

$$H = \begin{bmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & \vdots \\ 0 & -1 & 2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \\ 1 \end{bmatrix}$$

Tabel 4.9: Hasil SPL dengan matriks Hilbert

n	Metode	Solusi	Jenis Solusi
6	Eliminasi Gauss	• $x_0 = 3.000$ • $x_1 = 5.000$ • $x_2 = 6.000$ • $x_3 = 6.000$ • $x_4 = 5.000$ • $x_5 = 3.000$	tunggal
6	Gauss-Jordan	• $x_0 = 3.000$ • $x_1 = 5.000$ • $x_2 = 6.000$ • $x_3 = 6.000$ • $x_4 = 5.000$ • $x_5 = 3.000$	tunggal
6	Cramer	• $x_0 = 3.000$ • $x_1 = 5.000$ • $x_2 = 6.000$ • $x_3 = 6.000$ • $x_4 = 5.000$ • $x_5 = 3.000$	tunggal
6	Matriks Balikan	• $x_0 = 3.000$ • $x_1 = 5.000$ • $x_2 = 6.000$ • $x_3 = 6.000$ • $x_4 = 5.000$ • $x_5 = 3.000$	tunggal

Tabel 4.10: Hasil SPL dengan matriks Hilbert

10	Eliminasi Gauss	• $x_0 = 5.000$ • $x_1 = 9.000$ • $x_2 = 12.000$ • $x_3 = 14.000$ • $x_4 = 15.000$ • $x_5 = 15.000$ • $x_6 = 14.000$ • $x_7 = 12.000$ • $x_8 = 9.000$ • $x_9 = 5.000$	tunggal
10	Gauss-Jordan	• $x_0 = 5.000$ • $x_1 = 9.000$ • $x_2 = 12.000$ • $x_3 = 14.000$ • $x_4 = 15.000$ • $x_5 = 15.000$ • $x_6 = 14.000$ • $x_7 = 12.000$ • $x_8 = 9.000$ • $x_9 = 5.000$	tunggal

Tabel 4.11: Hasil SPL dengan matriks Hilbert

10	Cramer	• $x_0 = 5.000$ • $x_1 = 9.000$ • $x_2 = 12.000$ • $x_3 = 14.000$ • $x_4 = 15.000$ • $x_5 = 15.000$ • $x_6 = 14.000$ • $x_7 = 12.000$ • $x_8 = 9.000$ • $x_9 = 5.000$	tunggal
10	Matriks Balikan	• $x_0 = 5.000$ • $x_1 = 9.000$ • $x_2 = 12.000$ • $x_3 = 14.000$ • $x_4 = 15.000$ • $x_5 = 15.000$ • $x_6 = 14.000$ • $x_7 = 12.000$ • $x_8 = 9.000$ • $x_9 = 5.000$	tunggal

Analisis: Keempat metode menghasilkan solusi yang sama pada kedua ukuran dan cocok dengan bentuk tertutup $x_i = \frac{i(n+1-i)}{2}$ ($i = 1, \dots, n$). Matriks ini berkondisi baik (bilangan kondisi ≈ 19 untuk $n = 6$ dan ≈ 48 untuk $n = 10$, tumbuh sekitar n^2), sehingga galat pembulatan tidak membesar dan perbedaan antarmetode tidak terlihat pada 3 desimal. Sebagai pembanding, matriks Hilbert yang sebenarnya memiliki bilangan kondisi sekitar 1.5×10^7 ($n = 6$) dan 1.6×10^{13} ($n = 10$), sehingga kehilangan sekitar 7 dan 13 digit akurasi. Pada matriks seperti itu, Cramer dan Matriks Balikan biasanya menghasilkan galat lebih besar daripada eliminasi. Untuk $n = 10$ Cramer harus menghitung 11 determinan 10×10 , sehingga

paling mahal.

4.4 Sistem Persamaan Linear Berbentuk Matriks Augmented

4.4.1 Kasus Uji 1

Input: Matriks dari test\spl_kasus-5.txt.

$$\left[\begin{array}{cccc|c} 0 & 1 & -1 & 2 & -1 \\ 2 & 0 & 1 & -1 & 3 \\ -1 & 2 & 0 & 1 & -3 \\ 1 & -1 & 2 & 0 & 9 \end{array} \right]$$

Tabel 4.12: Hasil SPL augmented Kasus Uji 1

Metode	Solusi	Jenis Solusi
Eliminasi Gauss	• $x_0 = 1.000$ • $x_1 = -2.000$ • $x_2 = 3.000$ • $x_3 = 2.000$	tunggal
Eliminasi Gauss-Jordan	• $x_0 = 1.000$ • $x_1 = -2.000$ • $x_2 = 3.000$ • $x_3 = 2.000$	tunggal
Kaidah Cramer	• $x_0 = 1.000$ • $x_1 = -2.000$ • $x_2 = 3.000$ • $x_3 = 2.000$	tunggal
Matriks Balikan	• $x_0 = 1.000$ • $x_1 = -2.000$ • $x_2 = 3.000$ • $x_3 = 2.000$	tunggal

Analisis: Sistem persegi nonsingular dengan solusi tunggal $(1, -2, 3, 2)$, dan keempat metode sepakat. Hasil ini sama dengan kasus SPL $Ax = b$ nomor 1 yang dihitung secara independen, sehingga pembacaan masukan dalam bentuk matriks augmented dan dalam bentuk A, b terpisah konsisten.

4.4.2 Kasus Uji 2

Input: Matriks dari test\spl_kasus-6.txt.

$$\left[\begin{array}{cccccc|c} 1 & 0 & 2 & -1 & 0 & 3 & -7 \\ 0 & 1 & -1 & 2 & 1 & 0 & 6 \\ 2 & -1 & 0 & 1 & 3 & -2 & 13 \\ 1 & 1 & 1 & 1 & 1 & 1 & 3 \end{array} \right]$$

Tabel 4.13: Hasil SPL augmented Kasus Uji 2

Metode	Solusi	Jenis Solusi
Eliminasi Gauss	• $x_0 = 5.800 - a_0 - 1.600a_1$ • $x_1 = 2.600 - a_0 - 0.200a_1$ • $x_2 = -3.400 + a_0 + 0.800a_1$ • $x_3 = a_0$ • $x_4 = a_1$ • $x_5 = -2.000$	banyak
Eliminasi Gauss-Jordan	• $x_0 = 5.800 - 1.600a_0 - a_1$ • $x_1 = 2.600 - 0.200a_0 - a_1$ • $x_2 = -3.400 + 0.800a_1 + a_1$ • $x_3 = a_1$ • $x_4 = a_0$ • $x_5 = -2.000$	banyak
Kaidah Cramer	Error: SPLSolver.byCramer: Jumlah persamaan tidak sama dengan jumlah variabel!	tidak ada
Matriks Balikan	Error: SPLSolver.byInverse: Jumlah persamaan tidak sama dengan jumlah variabel!	tidak ada

Analisis: Sistem memiliki 4 persamaan dan 6 variabel dengan $\text{rank}(A) = \text{rank}([A|b]) = 4$, sehingga solusinya tak hingga dengan 2 parameter bebas. Gauss dan Gauss-Jordan menghasilkan himpunan solusi yang setara. Perbedaan hanya pada pelabelan parameter (a_0 dan a_1 tertukar pada x_3 dan x_4), yang tidak memengaruhi himpunan solusi karena pemilihan variabel bebas bersifat arbitrer. Cramer dan Matriks Balikan tidak dapat diterapkan karena matriks tidak persegi.

4.5 Sistem Persamaan Linear Berbentuk Umum

4.5.1 Kasus Uji 1

Input: Matriks dari `test\spl_kasus-7.txt`.

$$3x_1 + 2x_2 - x_3 + 4x_4 = 5$$

$$x_1 - 3x_2 + 2x_3 + x_4 = -4$$

$$4x_1 + x_2 + 5x_3 - 2x_4 = 7$$

$$2x_1 - x_3 + 3x_4 + x_4 = 3$$

Tabel 4.14: Hasil SPL umum Kasus Uji 1

Metode	Solusi	Jenis Solusi
Eliminasi Gauss	• $x_0 = -0.114$ • $x_1 = 1.841$ • $x_2 = 1.432$ • $x_3 = 0.773$	tunggal
Eliminasi Gauss-Jordan	• $x_0 = -0.114$ • $x_1 = 1.841$ • $x_2 = 1.432$ • $x_3 = 0.773$	tunggal
Kaidah Cramer	• $x_0 = -0.114$ • $x_1 = 1.841$ • $x_2 = 1.432$ • $x_3 = 0.773$	tunggal
Matriks Balikan	• $x_0 = -0.114$ • $x_1 = 1.841$ • $x_2 = 1.432$ • $x_3 = 0.773$	tunggal

Analisis: Sistem persegi nonsingular dengan solusi tunggal, dan keempat metode sepakat. Karena solusinya bukan bilangan bulat, kebenarannya diverifikasi dengan substitusi balik ke tiap persamaan (residu orde 10^{-15}). Perbedaan antarmetode, bila ada, hanya muncul pada digit di luar 3 desimal akibat urutan operasi dan pembulatan yang berbeda.

4.5.2 Kasus Uji 2

Input: Matriks dari test\spl_kasus-8.txt.

$$x_1 + x_2 + x_3 = 10.00$$

$$x_4 + x_5 + x_6 = 12.00$$

$$x_7 + x_8 + x_9 = 9.00$$

$$0.03521(x_1 + x_4 + x_7) + 0.68(x_2 + x_6) + 0.55432x_9 = 8.42$$

$$0.87654(x_1 + x_4 + x_7) + 0.30(x_3 + x_5 + x_8) = 11.05$$

$$0.03521(x_1 + x_4 + x_7) + 0.68(x_3 + x_5) + 0.55432x_2 = 6.19$$

$$x_1 + x_5 + x_9 = 11.00$$

$$x_2 + x_4 + x_8 = 8.50$$

$$x_3 + x_6 + x_7 = 11.50$$

Tabel 4.15: Hasil SPL umum Kasus Uji 2

Metode	Solusi	Jenis Solusi
Eliminasi Gauss	• $x_0 = 5.781 - 0.462a_0$ • $x_1 = 4.399 - 0.387a_0$ • $x_2 = -0.180 + 0.849a_0$ • $x_3 = -0.725 + 0.971a_0$ • $x_4 = 5.219 - 0.538a_0$ • $x_5 = 7.506 - 0.433a_0$ • $x_6 = 4.174 - 0.416a_0$ • $x_7 = 4.826 - 0.584a_0$ • $x_8 = a_0$	banyak
Eliminasi Gauss-Jordan	• $x_0 = 5.781 - 0.462a_0$ • $x_1 = 4.399 - 0.387a_0$ • $x_2 = -0.180 + 0.849a_0$ • $x_3 = -0.725 + 0.971a_0$ • $x_4 = 5.219 - 0.538a_0$ • $x_5 = 7.506 - 0.433a_0$ • $x_6 = 4.174 - 0.416a_0$ • $x_7 = 4.826 - 0.584a_0$ • $x_8 = a_0$	banyak
Kaidah Cramer	Error: SPLSolver.byCramer: Matriks persamaan tidak mempunyai determinan!	tidak ada
Matriks Balikan	Error: Matrix.ginv: Matriks tidak punya invers!	tidak ada

Analisis: Sistem memiliki 9 persamaan dan 9 variabel, tetapi $\text{rank}(A) = \text{rank}([A|b]) = 8$, sehingga matriks singular namun sistem konsisten dengan 1 parameter bebas. Ketergantungan linear ini wajar karena jumlah persamaan baris 1–3 sama dengan jumlah persamaan kolom 7–9 (keduanya menjumlahkan seluruh x_i). Gauss dan Gauss-Jordan menghasilkan solusi parametrik yang identik. Cramer dan Matriks Balikan gagal karena $\det A = 0$, padahal solusi ada. Ini kembali menunjukkan bahwa keduanya hanya berlaku untuk sistem persegi nonsi-

ngular. Koefisien desimal berpresisi 5 angka membuat pembulatan pada bentuk parametrik tampak pada digit ketiga, tetapi tidak mengubah struktur solusi.

4.6 Aplikasi Sistem Persamaan Linear

Diberikan dataset berukuran kecil yang berisi informasi tentang luas rumah dalam ratusan m² dan harga dalam ratusan juta

Tabel 4.16: Hasil aplikasi SPL pada model reaktor

X (luas rumah)	Y (harga)
1	3
2	5
3	6

Misalkan kita ingin membuat sebuah model prediksi harga rumah terhadap luas rumah berbentuk

$$y = \theta_0 + \theta_1 x.$$

Sekarang, perhatikan juga bahwa matriks-matriks yang bersesuaian dengan model yang ingin kita bentuk memiliki persamaan

$$Y = \theta X.$$

Kita akan mencari parameter θ_0 dan θ_1 menggunakan normal equation

$$\theta = (X^T X)^{-1} X^T Y.$$

Dengan demikian, kita mendapatkan sistem persamaan:

$$\begin{aligned} 3\theta_0 + (x_1 + x_2 + x_3)\theta_1 &= y_1 + y_2 + y_3 \\ (x_1 + x_2 + x_3)\theta_0 + (x_1^2 + x_2^2 + x_3^2)\theta_1 &= x_1 y_1 + x_2 y_2 + x_3 y_3 \end{aligned}$$

Tentukan nilai θ_0 dan θ_1 yang memenuhi persamaan linear tersebut.

Persamaan tersebut dapat kita ubah dengan menginputkan masing masing x dan y pada tabel ke x dan y pada persamaan sehingga diperoleh

$$\begin{aligned} 3\theta_0 + 6\theta_1 &= 14 \\ 6\theta_0 + 14\theta_1 &= 31 \end{aligned}$$

Sehingga diperoleh hasil akhir menggunakan metode Gauss

$$\begin{aligned} \theta_0 &= 1.667 \\ \theta_1 &= 1.500 \end{aligned}$$

Analisis: Nilai tersebut sesuai dengan solusi eksak $\theta_0 = 5/3$ dan $\theta_1 = 3/2$. Model $\hat{y} = 1.667 + 1.500x$ berarti kenaikan luas rumah sebesar 100 m² berkaitan dengan kenaikan harga sekitar 150 juta rupiah. Nilai prediksi untuk ketiga data adalah 3.167, 4.667, dan 6.167 dengan residu -0.167 , 0.333 , dan -0.167 (jumlah residu 0, ciri regresi dengan intersep). Sistem 2×2 ini berkondisi baik, sehingga tidak ada galat numerik yang berarti. Dengan hanya 3 data, model ini hanya ilustrasi dan tidak cukup kuat untuk generalisasi.

4.7 Interpolasi Polinomial

4.7.1 Studi Kasus 1

Gunakan tabel di bawah ini untuk mencari polinomial interpolasi yang melewati titik-titik berikut:

x	0.2	0.4	0.6	0.8	1.0	1.2	1.4
$f(x)$	0.182	0.336	0.470	0.588	0.693	0.788	0.875

Kemudian, tentukan juga nilai-nilai dari $f(x)$ dari polinomial yang telah dikonstruksi untuk nilai-nilai berikut:

- $x = 0.3$
- $x = 0.55$
- $x = 0.95$
- $x = 1.38$

x	0.3	0.55	0.95	1.38
$f(x)$	0.262	0.438	0.668	0.867

Analisis: Keempat nilai sesuai dengan nilai ekspektasi pada soal. Seluruh titik uji berada dalam domain data $[0.2, 1.4]$, sehingga ini adalah interpolasi, bukan ekstrapolasi. Data tampak berasal dari $\ln(1+x)$ yang dibulatkan ke 3 desimal. Selisih program terhadap $\ln(1+x)$ maksimum sekitar 4×10^{-4} , yang berasal dari pembulatan data, bukan dari galat metode. Polinom derajat 6 membentuk sistem Vandermonde berukuran 7×7 . Untuk jumlah titik yang jauh lebih banyak, matriks ini cepat memburuk kondisinya dan polinom dapat berosilasi (fenomena Runge).

4.7.2 Studi Kasus 2

Seorang insinyur sedang melakukan pengujian terhadap sebuah komponen elektronik untuk mengukur bagaimana suhu komponen meningkat seiring bertambahnya waktu ketika dialiri oleh arus listrik terus-menerus. Data suhu (T dalam $^{\circ}\text{C}$) yang tercatat pada beberapa waktu pengukuran (t , dalam menit) adalah sebagai berikut :

t (menit)	0	2	4	6	8	10
T	25.0	31.2	38.6	47.5	58.1	70.8

Insinyur tersebut ingin mengetahui perkiraan suhu komponen pada waktu-waktu tertentu di antara titik pengukuran, tanpa harus melakukan pengujian ulang. Buatlah program yang menerima masukan nilai t dan menghitung perkiraan nilai T menggunakan interpolasi polinomial. Lakukan pengujian pada nilai-nilai berikut

- $t = 1 \quad T = 27.981$
- $t = 3.5 \quad T = 36.617$
- $t = 7 \quad T = 52.571$

- $t = 9.2 \text{ } T = 65.426$

Analisis: Keempat nilai sesuai dengan nilai referensi pada soal. Seluruh t uji berada di dalam $[0, 10]$, sehingga hasilnya adalah interpolasi yang valid. Data bersifat mulus dan monoton naik dengan laju kenaikan yang membesar, sehingga polinom tidak beresilasi dan nilai antar titik wajar. Polinom hanya layak dipakai di dalam rentang pengukuran dan tidak untuk ekstrapolasi.

4.8 Interpolasi Splina Bézier Kubik

Konstruksikan kurva natural cubic spline yang melewati 5 titik data berikut.

i	0	1	2	3	4
x_i	1	2	3	4	5
y_i	2	3	4	5	6

Turunan kedua: $M_0 = M_1 = M_2 = M_3 = M_4 = 0.000$

Persamaan tiap segmen (setelah disederhanakan):

$$S_0(x) = S_1(x) = S_2(x) = S_3(x) = x + 1$$

Domain: $[1, 2)$, $[2, 3)$, $[3, 4)$, $[4, 5]$

Evaluasi:

x	$S(x)$
1.5	2.5
2.5	3.5
3.5	4.5
4.5	5.5

Analisis: Kelima titik kolinear ($y = x + 1$), sehingga semua turunan kedua $M_i = 0$ dan setiap segmen menjadi garis yang sama. Ini merupakan uji kewarasan yang baik karena spline kubik natural harus mereproduksi fungsi linear secara eksak. Semua nilai evaluasi sesuai dengan $x + 1$. Karena sistem untuk M_i berbentuk tridiagonal dengan ruas kanan nol, solusinya nol tanpa galat pembulatan yang berarti.

4.9 Regresi Polinomial Berganda

Diberikan 7 titik data observasi

i	0	1	2	3	4	5	6
x_i	-3	-2	-1	0	1	2	3
y_i	19,9	4,4	-0,5	2	6,5	15,6	38,1

Konstruksikan model kurva regresi spline kubik untuk memetakan hubungan antara variabel x dan y dengan menetapkan satu titik simpul (knot) internal pada $\xi_1 = 0$.

Hasil perhitungan program:

Koefisien	Nilai
β_0	2.000
β_1	3.000
β_2	0.000
β_3	-1.000
β_4	2.000

Posisi knot internal: $\xi_1 = 0$, dengan nilai prediksi di knot $\hat{y}(0) = 2.000$.

Model regresi spline kubik yang terbentuk:

$$y = 2.000 + 3.000x + 0.000x^2 - 1.000x^3 + 2.000(x-0)_+^3.$$

Karena $(x-0)_+ = \max(0, x)$, model dapat dituliskan secara *piecewise*:

$$y = \begin{cases} 2 + 3x - x^3, & x \leq 0, \\ 2 + 3x + x^3, & x > 0. \end{cases}$$

Perbandingan nilai aktual dan nilai prediksi:

x_i	y_i (aktual)	$\hat{y}_i$ (prediksi)	Galat
-3	20.000	20.0	0
-2	4.000	4.0	0
-1	0.000	0.0	0
0	2.000	2.0	0
1	6.000	6.0	0
2	16.000	16.0	0
3	38.000	38.0	0

Analisis: Hasil yang didapatkan dari fungsi sesuai dengan test case yang diberikan dengan galat 0

4.10 Rekapitulasi Hasil

Tabel 4.17: Rekapitulasi hasil seluruh kasus uji

Fitur	Kasus	Status	Catatan
Determinan	Kasus 1–3	berhasil	Kofaktor dan OBE sepakat; -0.000 pada Kasus 2 adalah galat pembulatan
Invers	Kasus 1–2	berhasil	Kasus 2 singular dan terdeteksi tanpa <i>crash</i>
SPL $Ax = b$	Kasus 1–4	berhasil	Kasus 2 parametrik; Kasus 3 tak konsisten; Cramer/Balikan tidak berlaku pada sistem non-persegi atau singular
SPL Augmented	Kasus 1–2	berhasil	Kasus 2 parametrik (2 parameter bebas)
SPL Umum	Kasus 1–2	berhasil	Kasus 2 singular, 1 parameter bebas
Aplikasi SPL	Harga rumah	berhasil	$\theta_0 = 1.667$, $\theta_1 = 1.500$ sesuai solusi eksak
Interpolasi Polinomial	Studi Kasus 1–2	berhasil	Seluruh nilai uji cocok dengan referensi
Splina Kubik Natural	5 titik	berhasil	Data kolinear, spline tereduksi menjadi garis
Regresi Spline Kubik	7 titik	berhasil	Sesuai dengan test case

Bab 5

Penutup

5.1 Kesimpulan

Tugas besar ini berhasil menghasilkan pustaka aljabar linier mandiri berbasis Java bernama NurEngine, yang dibangun tanpa *library* perhitungan matriks, regresi, atau interpolasi instan. Pustaka terdiri atas lima modul inti (Matrix, SPLSolver, Interpolation, Regression, dan ImageHoleFill) serta dua antarmuka, yaitu CLI (App.java) dan GUI berbasis JavaFX (GUI.java). Seluruh fitur yang dipersyaratkan telah diimplementasikan, yaitu penyelesaian SPL dengan empat metode (Eliminasi Gauss, Gauss-Jordan, Kaidah Cramer, dan Matriks Balikan), determinan (ekspansi kofaktor dan reduksi baris), invers matriks (augment dan adjoint), interpolasi polinomial, Natural Cubic Spline Interpolation, regresi spline kubik dengan *truncated power basis*, serta *Image Inpainting* berbasis Persamaan Laplace dengan metode Gauss-Seidel sebagai fitur bonus.

5.1.1 Kelebihan Sistem

- **Arsitektur modular dengan dependensi yang jelas.** Matrix dan ImageHoleFill berdiri sendiri, sedangkan SPLSolver, Interpolation, dan Regression tersusun berlapis. Struktur ini memudahkan pengujian per modul dan pengerjaan paralel oleh anggota kelompok.
- **Dua antarmuka dengan logika yang sama.** CLI dan GUI JavaFX memakai fungsi pustaka yang sama, sehingga perilaku keduanya konsisten.
- **Proses *build* yang otomatis.** Maven memungkinkan kompilasi dan eksekusi dengan satu perintah (`mvn clean package`).

5.1.2 Kekurangan Sistem

- **Presisi Data** Karena presisi tipe data double mentok di 15-17 angka di belakang desimal, beberapa algoritma (seperti SPLSolver.byInverse) akan hancur untuk matriks besar ataupun matriks ill-conditioned
- **Tampilan GUI sederhana.** GUI yang dibuat dimaksimalkan untuk fungsi, sehingga menggunakan desain yang simpel
- **Pembulatan** Output hasil yang diberikan dibulatkan 3 titik di belakang koma, sehingga tidak terlalu presisi

- **Kerapihan** Output matriks tidak terlalu rapi. Ini dilakukan untuk meningkatkan konsistensi

5.2 Refleksi Anggota Kelompok

5.2.1 Rionaldo Casey Pandhitha (13525030)

oop dan java adalah pengalaman baru yang menyenangkan

5.2.2 Raffi Fauzi Hermawan (13525054)

Tubes yang lumayan asik, meski banyak debugging karena bahasa yang baru pertama kali dipakai, cuma karena sintaksnya mirip c mayan lancar dan juga ilmu baru mengenai error handling dan input handling. Setelah itu goat Fachry Azriel aka Forreal jago bagi tugas yang membuat tidak ada terjadi momen "saling nunggu." Last but also least, 5 poin untuk latex tidak se-worth it itu karena sangat "nguli"

5.2.3 Fachry Azriel Fajdwani (13525110)

sejauh ini, ini tubes ngoding yang paling asik, nggak kayak alpro yang terlalu "komputer". pengerjaan SPLSolver juga bikin aku terpaku depan laptop sampe jam 3, muter otak gimana caranya representasiin output splsolver ini tanpa harus pake HashMap, Map, dan tipe bentukan lain kayak sebelah

5.2.4 Refleksi Bersama

Kami belajar mengenai cara pembagian tugas yang baik dan benar, selain itu kami juga belajar bahasa baru dan untuk mengerjakan tugas jauh sebelum deadline. Sebenarnya, hari Rabu kemarin kami bisa mengumpulkan tugasnya, namun karena extend maka kami menambahkan GUI

Daftar Pustaka

- [1] R. Munir, “Matriks Eselon,” Bahan Kuliah IF2123 Aljabar Linier dan Geometri, STEI-ITB, Seri Bahan Kuliah Algeo #2, 2025. <https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2025-2026/Algeo-02-Matriks-Eselon-2025.pdf>
- [2] R. Munir, “Sistem Persamaan Linier (SPL),” Bahan Kuliah IF2123 Aljabar Linier dan Geometri, STEI-ITB, Seri Bahan Kuliah Algeo #3, 2023. <https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2025-2026/Algeo-03-Sistem-Persamaan-Linier-2025.pdf>
- [3] R. Munir, “Tiga Kemungkinan Solusi SPL,” Bahan Kuliah IF2123 Aljabar Linier dan Geometri, STEI-ITB, Seri Bahan Kuliah Algeo #4, 2025. <https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2025-2026/Algeo-04-Tiga-Kemungkinan-Solusi-SPL-2025.pdf>
- [4] R. Munir, “Sistem Persamaan Linier (SPL): Metode Eliminasi Gauss-Jordan,” Bahan Kuliah IF2123 Aljabar Linier dan Geometri, STEI-ITB, Seri Bahan Kuliah Algeo #5, 2025. <https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2025-2026/Algeo-05-Sistem-Persamaan-Linier-2-2025.pdf>
- [5] R. Munir, “Determinan Bagian 1,” Bahan Kuliah IF2123 Aljabar Linier dan Geometri, STEI-ITB, Seri Bahan Kuliah Algeo #8, 2025. <https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2025-2026/Algeo-08-Determinan-bagian1-2025.pdf>
- [6] R. Munir, “Determinan Bagian 2,” Bahan Kuliah IF2123 Aljabar Linier dan Geometri, STEI-ITB, Seri Bahan Kuliah Algeo #9, 2025. <https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2025-2026/Algeo-09-Determinan-bagian2-2025.pdf>
- [7] R. Munir, *Metode Numerik*, Bab 4, “Solusi Sistem Persamaan Lanjar.” <https://informatika.stei.itb.ac.id/~rinaldi.munir/Buku/Metode%20Numerik/pdf/BAb-%2004%20Solusi%20Sistem%20Persamaan%20Lanjar.pdf>
- [8] R. Munir, *Metode Numerik*, Bab 5, “Interpolasi dan Regresi.” <https://informatika.stei.itb.ac.id/~rinaldi.munir/Buku/Metode%20Numerik/pdf/BAb-%2005%20Interpolasi%20Polinom.pdf>
- [9] Laboratorium Ilmu dan Rekayasa Komputasi, “Spesifikasi Tugas Besar 1 IF2123 Aljabar Linier dan Geometri: Sistem Persamaan Linier, Determinan, dan Penerapannya,” ver. 1.1, Institut Teknologi Bandung, 2026.

- [10] L. Leal, “Numerical Interpolation: Natural Cubic Spline,” *Towards Data Science Archive*, Dec. 2, 2018. <https://medium.com/data-science/numerical-interpolation-natural-cubic-spline-52c1157b98ac>
- [11] E. W. Weisstein, “Cubic Spline,” *MathWorld - A Wolfram Web Resource*. <https://mathworld.wolfram.com/CubicSpline.html>
- [12] R. Hiptmair, “Normal Equation Methods,” *Numerical Methods for Computational Science and Engineering*, ETH Zurich, 2025. https://people.math.ethz.ch/~grsam/NumMeth/Parts/NCSEFL_handout_chapter_3.pdf
- [13] S. Eddins, “Region filling and Laplace’s equation,” *MathWorks Blogs*, Jun. 17, 2015. <https://blogs.mathworks.com/steve/2015/06/17/region-filling-and-laplaces-equation/>
- [14] J. Demmel, “MA221 Lecture 12: Jacobi and Gauss-Seidel,” University of California, Berkeley, 2022. https://people.eecs.berkeley.edu/~demmel/ma221_Spr22/Lectures/Lecture_12.pdf

Bab 6

Lampiran